



David Ge

Department of Computer Science,
Columbia University,
New York, NY 10027
e-mail: Dg3517@columbia.edu

Hao Ji¹

Center for Advanced Research Computing,
University of Southern California,
Los Angeles, CA 90089
e-mail: haoji@usc.edu

Tsung-Jui Wu

Department of Computer Science,
University of Southern California,
Los Angeles, CA 90089
e-mail: tsungjui@usc.edu

Bingling Huang

Department of Mechanical Engineering,
California State University, Fullerton,
Fullerton, CA 92831
e-mail: bihuang@fullerton.edu

Quanchao Lu

School of Computer Science,
Georgia Institute of Technology,
Atlanta, GA 30332
e-mail: qlu43@gatech.edu

Shuo Shi

Michtom School of Computer Science,
Brandeis University,
Waltham, MA 02453
e-mail: sshi.contact@gmail.com

Efficient Training in Multiagent Reinforcement Learning: A Communication-Free Framework for the Box-Pushing Problem

Effective coordination is a critical challenge in the design of self-organizing systems (SOSs), particularly in decentralized training where explicit communication incurs high costs. Traditional approaches often rely on inter-agent communication, but this can introduce substantial system overhead. The challenge becomes even more pronounced in multiagent reinforcement learning (MARL)-based systems, where the training process happens in an end-to-end black-box manner. To address this issue, we explore alternative methods to enhance the exploration phase without relying on direct communication, thereby improving search efficiency. Therefore, the shared pool of information (SPI) is proposed in this article, which is a communication-free framework designed to provide agents with structured shared information at initialization. By offering a common foundation for exploration, SPI helps guide group action choices and facilitates more effective decision-making. This approach enables agents to learn coordinated behaviors without bringing the high costs associated with continuous communication. The efficiency of SPI is assessed and validated in the box-pushing problem, a task that requires agents to collaboratively maneuver a box toward a goal while avoiding obstacles. Our findings indicate that SPI accelerates learning, enhances coordination, increases success rates, and optimizes trajectory optimization. These results highlight SPI as a promising and scalable approach for scenarios where communication is costly or infeasible.

[DOI: 10.1115/1.4071239]

Keywords: multiagent reinforcement learning, box-pushing problem, communication-free framework, training efficiency, cooperative learning, artificial intelligence, engineering informatics, knowledge engineering, machine learning for engineering applications

1 Introduction

Self-organizing systems (SOSs) are well suited to address scalability, bottlenecks, and reliability issues that are typically encountered in centralized systems [1], thanks to their decentralized and localized organization mechanisms. Learning-based methods, specifically multiagent reinforcement learning (MARL), are commonly used to enable agents in SOSs to learn through trial-and-error interactions with the environment. Although MARL excels in offering a design that avoids conceptualizing every possible incident or detail [2], cooperation and coordination in a multiagent system have been difficult, yet crucial problems to solve.

Many studies have explored how to wisely coordinate individuals in a team. For example, Li et al. [3] proposed a cooperative

contextual reinforcement learning model to address the lack of cooperation between couriers while also considering the system context. Liu et al. [4] proposed a cooperative reinforcement learning (RL) algorithm for distributed economic dispatch in microgrids. Despite these advancements, our understanding of underlying mechanisms of self-organized coordination remains limited. Thus, further investigation is required to facilitate better design for complex tasks in engineering.

To address this issue, we proposed a novel team exploration mechanism, shared pool of information (SPI), integrated into a MARL communication-free framework. SPI serves to guide the decision-making of individual agents based on a global predefined group behavior pattern. Such patterns, which are articulated by simple probability distributions, can help regulate individual actions to achieve a more balanced collective behavior. The idea is borrowed from the *knowledge base* in human organizations [5].

The primary goal of integrating SPI into the system is to promote implicit coordination, enabling agents to collaborate

¹Corresponding author.

Manuscript received October 5, 2025; final manuscript received January 31, 2026; published online March 17, 2026. Assoc. Editor: Hongyue Sun.

effectively without directly exchanging information. In this work, communication-free learning refers to the absence of inter-agent communication during execution, meaning that each agent's behavior is not influenced by any other agents' behavior. While SPI initializes agents from an SPI, the agents coordinate in a decentralized mechanism.

SPI enables guided learning rather than pure random exploration, and it provides additional information without introducing a communication overhead. Those make it a more efficient approach that mitigates the risks of learning from unbalanced data [6]. It is particularly important for MARL training, as previous work has demonstrated that communication can be costly and risky for training [2,7].

To evaluate the effectiveness of our approach, we select box-push problems as experimental scenarios. In this task, a team of agents collaboratively pushes a box towards a designated goal while avoiding any collisions. This scenario represents some applications in manufacturing warehouse and production line, where effective coordination is crucial.

Successful execution of this task requires a high level of team intelligence for several reasons. First, agents must cooperatively control the movement and orientation of the box, coordinate and plan globally for the group goal. In addition, they must accomplish this using localized information. Previous research has shown that agents with restricted observations would experience action conflicts, such as pushing against each other [2,7]. Although agents can address this issue through implicit observations or direct communication [8–10], incorporating communication adds extra complexity to the system [2].

In our experiments, we evaluate SPI's impact on training efficiency by comparing it to random exploration. The results demonstrate that SPI significantly improves coordination, reducing both conflicting actions and redundant agent movements. Such guidance makes each agents' actions contribute more meaningfully to the collective goal. This idea can be versatile and complement other algorithms to tackle coordination and cooperation issues in learning-based SOSs. Effectiveness is quantitatively measured by fitness tests, specifically an origin avoidance test and an angular spread test.

The rest of the article is organized as follows: Sec. 2 presents related work on this topic. Section 3 describes the methodology and experimental design. Section 4 offers the experimental results along with thorough analysis and discussion. Finally, the conclusion and future direction are presented in Sec. 5.

2 Related Work

2.1 MARL Techniques for the Box-Pushing Problem. The box-pushing problem has been extensively studied using both single-agent and multiagent RL. In simple environments, where there are few to no obstacles, research indicates that both methods can achieve comparable performance. However, in complex environment, MARL inevitably demands a higher degree of system capability compared to single-agent RL [11,12]. To facilitate the MARL training, several novel methodologies have been proposed. Parra-Gonzalez et al. proposed a path planning algorithm that minimize the number of reconfigurations, aiding robots in planning and optimizing routes for the box-pushing problem [13]. Other researchers have extended the standard Q-learning framework [14] to support MARL implementation [15,16]. To simplify multiagent system design, Yasuda et al. [16] proposed a clustering-based technique, where agents' learned behaviors are grouped into clusters, each governed by a specific set of rules to determine optimal actions at a given step. Hwang et al. [15] introduced an adaptive state aggregation Q-learning method, which improves coordination by partitioning the state space into manageable chunks. Additionally, Wang and de Silva [17] explored a hybrid approach that combines genetic algorithms with RL, using an arbitrator to evaluate outputs from both methods and to guide agent decision-making.

2.2 Centralized Training for Decentralized Execution.

Centralized training for decentralized execution (CTDE) is a widely used training paradigm in MARL. In this framework, agents are trained through global information (centralized learning) but make decisions independently based only on their local information during actual execution (decentralized execution) [18]. To improve coordination with CTDE, Fan et al. introduced f-MAT, which employs graph modeling to facilitate more efficient collaboration [19]. Centralized learning can improve performance by leveraging global information [20]. There are two main ways to achieve this, namely a single centralized critic shared by all agents, and a number of individual centralized critics assigned to each agent [21,22]. Several studies have explored variations of the designs of centralized critics. Kim et al. proposed SchedNet where a centralized critic provides structured feedback using message encoders, action selectors, and weight generators [23]. Chen proposed centralized training and exploration with decentralized execution, utilizing policy distillation as an example of a critic for each agent [24]. In addition, Vezhnevets et al. proposed a Bayesian mode to further optimize the exploration phase of CTDE training. It weighs the costs of the exploration against the expected benefit gained from the exploration [25]. However, in real-world applications, centralized models often struggle to collect complete global information [26].

2.3 Communication in Multiagent Systems.

MARL tasks have high requirements on agent team coordination capabilities. In a Markov decision process (MDP), each decision step depends on the joint action of all agents, leading to a large action space problem [27]. To overcome that issue, local optimization for the different agents through reward and value sharing [28,29], direct policy search methods [30], and factored MDPs [31,32] also been explored. To enhance coordination efficiency, Li et al. introduced coordinate graphs, which are dynamically generated structures that enable agents to share information and collaboratively determine joint actions in a structured manner [33]. These approaches collectively aim to improve the efficiency of MARL while addressing the challenges associated with coordination and communication overhead.

In summary, communication overhead represents another challenge in MARL systems, where agents must coordinate decisions while managing limited bandwidth and computational resources. Thus, to address communication costs, several methodologies have emerged across different MARL domains. Domain-specific solutions for box-pushing include path planning algorithms that minimize reconfigurations [13], extensions of Q-learning frameworks for multiagent settings [14–16], clustering-based behavioral grouping, adaptive state aggregation methods [15], and hybrid genetic algorithm-RL approaches [17]. While these methods improve coordination, they typically assume reliable communication channels and do not explicitly optimize for communication efficiency, which is a gap our work addresses through adaptive communication strategies applicable across various MARL environments.

In this work, we use the box-pushing problem as a representative testbed to evaluate our approach. Box-pushing provides an ideal benchmark because it requires tight coordination between agents to succeed, making communication quality directly observable in task performance, while its scalable complexity allows systematic evaluation of how communication strategies perform as the number of agents and environmental constraints increase. Furthermore, the discrete action space and clear success metrics enable a rigorous quantitative analysis of the trade-offs between communication overhead and coordination effectiveness.

3 Methodology

Building on the fully cooperative MARL framework using deep Q-learning [34], we developed an implicit coordination mechanism

called the SPI, which is integrated into the multiagent learning process, to provide agents with a shared reference for individual decision-making, enhancing cooperation and promoting more cohesive team behaviors.

3.1 Environment. Figure 1 shows the environment, which consists of outer boundary walls, multiple circular obstacles, square-shaped agents, a rectangular box, and a diamond-shaped goal. The environment is constructed by *Pygame* library [35]. The graphical illustration demonstrates a possible state of the box-pushing task in which agents collaboratively push the box. Agents need to navigate the box toward the goal area, while simultaneously avoiding collisions with obstacles or surrounding walls. The task is terminated once collisions occur or the box fails to reach the goal in 300 steps.

3.2 Action Space and State Space. At each time-step, each agent selects one of the six regions around the box to apply forces (see Fig. 2), directing the movement and orientation of the box accordingly. Multiple agents can select the same box region. Thus, the action space A can be represented as follows:

$$A = \{a_1, a_2, a_3, a_4, a_5, a_6\} \quad (1)$$

We use a binary indicator (1/0) to represent whether a specific region has been selected by an agent. For example, in Fig. 2, if the agent selects Region 1, the corresponding action space A is defined as $\{1, 0, 0, 0, 0, 0\}$.

A sensor is assumed to be mounted on the box to detect environmental information within a range of 150-pixel radius. The information includes the angle between the box and the goal, as well as the presence of obstacles in specific surrounding areas. The data are then transformed into the state space S , represented as follows:

$$S = \{s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9\} \quad (2)$$

In the above state space, $s_1, s_2, \dots, s_8$ represent the eight octants in Fig. 3. Each digit is the presence or absence of obstacles or wall. s_9 represents the angle between the box and the goal, scaling in the range $[-1, 1]$. An example is presented in Fig. 3. In that case, the state information can be represented as

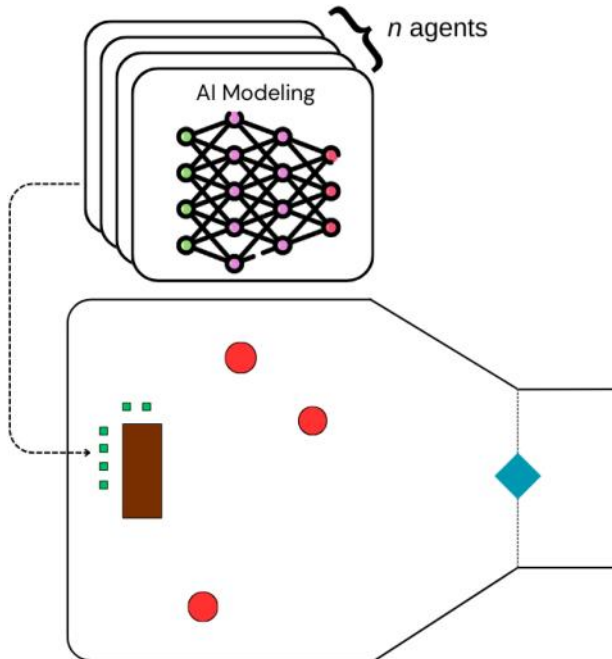


Fig. 1 Environment of box-pushing task

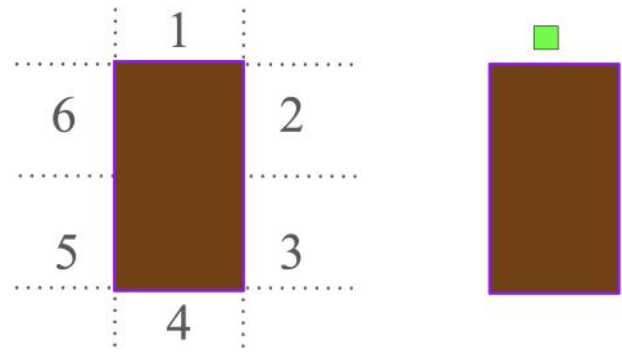


Fig. 2 Action space

$S = \{0, 0, 1, 1, 1, 1, 0, 0, -0.83\}$ since the angle $\theta = 30^\circ$ translates to $s_9 = (\theta - 180)/180 = -0.83$.

3.3 Reward Schema and Model Training. To enable multiple agents to self-organize and collaboratively transport the box to the designated target area, a carefully structured reward function is essential for effective agent training. Building upon established Q-table-based reward schemes for box-pushing scenarios [11,12], we propose a novel reward architecture comprising four distinct components: distance traveled, rotation, collision, and goal achievement bonuses. Given the homogeneous nature of the agents and their cooperative objective in box transportation, uniform reward distribution across all agents is implemented during the training phase to ensure coordinated learning and behavior emergence.

3.3.1 Distance Reward. (R_{dis}) is the reward to encourage agent teams to move the box approaching to the goal. D_{old} and D_{new} represent the distance in pixels between the box's sensor and the goal at the previous and current time-steps, respectively:

$$R_{dis} = (D_{old} - D_{new}) \times 2.5 \quad (3)$$

3.3.2 Rotation Reward. (R_{rot}) is the reward designed to discourage excessive rotation of the box in a single time-step. α_{old} and α_{new} represent angles between the box's x -axis and the goal at the previous and current time-steps, respectively:

$$R_{rot} = \cos(\alpha_{new} - \alpha_{old}) - 0.98 \quad (4)$$

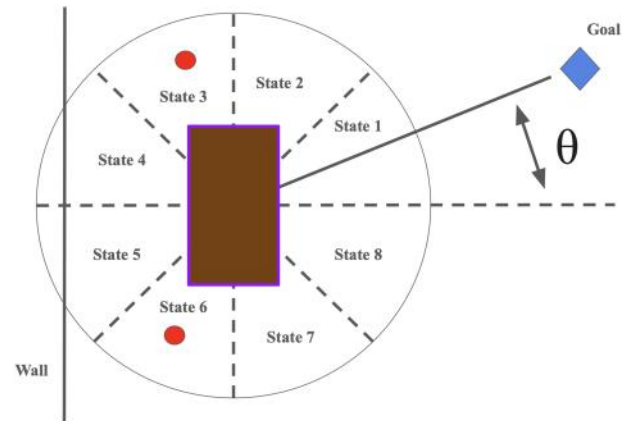


Fig. 3 State space

3.3.3 Collision Reward. (R_{col}) is the negative reward applied if the box collides with an obstacle or wall:

$$R_{\text{col}} = \begin{cases} 0 & \text{if there is no collision} \\ -900 & \text{if there is a collision} \end{cases} \quad (5)$$

3.3.4 Goal Reward. (R_{goal}) is the positive reward assigned once the task has been completed successfully:

$$R_{\text{goal}} = \begin{cases} 900 & \text{if goal is reached} \\ 0 & \text{if goal is not reached} \end{cases} \quad (6)$$

The total reward is formulated as a weighted sum of distance, rotation, collision, and goal rewards. All reward weights are set as hyperparameters based on prior trainings [36]. The values are defined in Table 1, where $\omega_1 + \omega_2 + \omega_3 + \omega_4 = 1$:

$$R_{\text{Total}} = \omega_1 R_{\text{dis}} + \omega_2 R_{\text{rot}} + \omega_3 R_{\text{col}} + \omega_4 R_{\text{goal}} \quad (7)$$

3.4 Speed Factor. The speed factor is designed to control how much force agents can exert. This affects how much agents can both displace and rotate the box. In this experiment, the speed factor is reduced to one-half of its original value stated in Ref. [36], effectively limiting the agent's pushing strength. With reduced force, individual agents are less capable of moving the box efficiently on their own. Consequently, cooperation becomes crucial, as the lack of collaboration results in slower progress. These altered dynamics are intended to evaluate how agents adapt to a more challenging environment where teamwork is essential to achieve the desired outcome.

3.5 Rules of Designing Shared Pool of Information. The primary goal of integrating SPI into the system is to promote implicit coordination, enabling agents to collaborate effectively without directly exchanging information.

Therefore, we proposed hypotheses and rules for the designed SPI:

- (1) All agents have access to the same shared information at the beginning.
- (2) Agents can only observe or read from the shared information.
- (3) The shared information remains unmodifiable by the agents.

These rules ensure that agents are identical at instantiation and operate without requiring direct communication. SPI does not eliminate uncertainty arising from partial observability, instead, it mitigates its effects by introducing shared inductive biases through a common, structured initialization. Although agents rely

solely on local observations and do not communicate during execution, this shared prior promotes compatible behaviors and facilitates the emergence of cooperation under limited sensory inputs. As a result, SPI remains applicable under standard MARL partial-observability conditions without requiring explicit inter-agent communication.

Overall, the integration of SPI remains efficient and lightweight, introducing minimal computational overhead.

3.6 Shared Pool of Information. The SPI enables implicit coordination in multiagent systems through a two-component framework: a “map” and a “key.” The map consists of probability distribution lists (PDLs) that encode idealized collective behaviors—essentially representing how agents should distribute their actions if acting as a unified entity. To maintain decentralized decision-making, each agent accesses this shared map through a randomly generated “key” that serves as an index, directing them to a specific PDL at each time-step. Since all agents use the same key simultaneously but select actions probabilistically according to the retrieved distribution, they achieve coordinated exploration without direct communication. This mechanism transforms pure random exploration into guided pseudo-random exploration, where agents independently make decisions yet collectively follow complementary action patterns that promote effective cooperation and balanced team behavior.

The *map* refers to a list of probability distributions that can be used to describe the ideal outcomes when all agents push the box simultaneously. It simulates the movement of the box under the assumption that all agents act as a single coordinated entity.

However, directly implementing such a distribution would fundamentally alter the problem from MARL to a single-agent RL approach. To preserve decentralized decision-making, SPI incorporates a “key” that allows agents to interpret the map without explicit communication.

The *key* serves as a randomized indexing mechanism that enable agents to access the “map” (the shared probability distribution). It must have sufficiently large range – exceeding the number of agents in the system – to ensure diversity in decision-making. Moreover, the key's distribution must also be uniform, preventing agents from developing biases toward specific distributions in the “map.” In the experiment, the “key” is generated using PYTHON's random library. New “key” will be re-generated every step of training which is available for all agents.

The “map” in SPI consists of numerous of PDLs—the exact number is discussed in Sec. 4.1.3. Since each agent has six possible actions, each PDL contains six entries, representing the probability of selecting each action. The sum of all entries in a PDL is exactly 1. Table 2 presents examples of valid and invalid PDLs.

Table 1 Hyperparameters in training; all hyperparameter values were selected based on previous experiments

Reward parameters	
ω_1	0.6
ω_2	0.1
ω_3	0.1
ω_4	0.2
Training parameters	
Replay memory size	1000
Mini-batch size	32
Discount factor	0.99
Learning rate α	0.001
Total training episodes	20,000
ϵ	1.0 $\rightarrow$ 0.005
Annealing steps	100,000
Target network update frequency	200

Table 2 Examples of valid and invalid PDL

PDL type	Examples	Explanation
Valid PDL	$\begin{bmatrix} \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6} \end{bmatrix}$ $[1, 0, 0, 0, 0, 0]$	6 dimensions with sum equals to 1
Invalid PDL	$\begin{bmatrix} \frac{1}{6}, \frac{1}{6}, \frac{1}{4}, \frac{1}{4}, \frac{1}{12}, \frac{1}{12} \end{bmatrix}$ $\begin{bmatrix} \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7}, \frac{1}{7} \end{bmatrix}$ $\begin{bmatrix} \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5} \end{bmatrix}$ $\begin{bmatrix} \frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4} \end{bmatrix}$	Sum = $\frac{6}{7} \neq 1$ Sum = $1.2 \neq 1$ Wrong dimension: 4 elements instead of 6

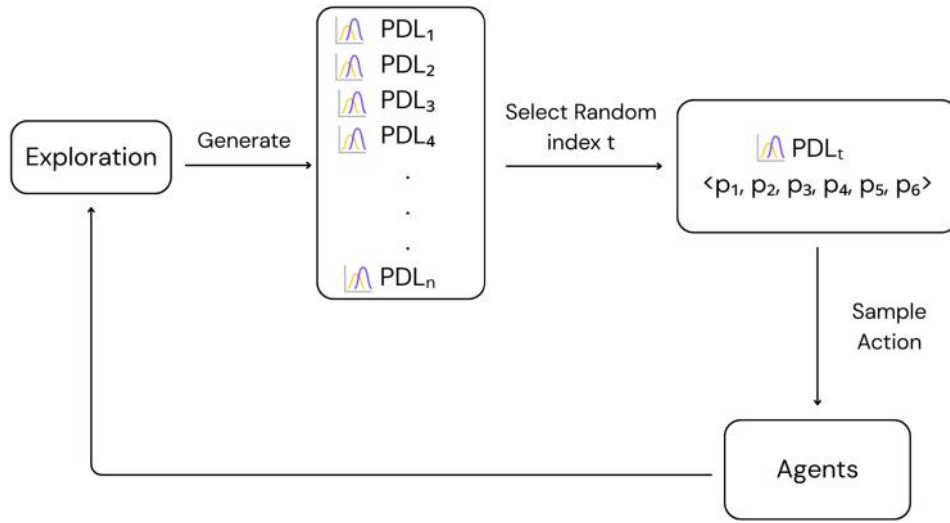


Fig. 4 Procedure of action selection

During exploration, agents use the “map” in the following way: (1) the agent reads the generated “key,” (2) the agent retrieves the corresponding PDL associated with the “key,” and (3) the agent selects an action based on the probabilities in the PDL, shown in Fig. 4.

3.7 Generating Probability Distribution List. As introduced in Fig. 2 in Sec. 3.2, the action space consists of six possible actions. However, due to their physical effects, actions 2 and 3, as well as actions 5 and 6, are functionally equivalent. To simplify the computation of the ideal distribution, the action space can be effectively reduced to four distinct actions. The procedure for generating a single PDL is outlined in the following parts.

3.7.1 Generating the Initial Values. The process utilizes a parameter; we sample four values from a uniform distribution $\mathcal{U}(0, 1)$:

$$u_i \sim \mathcal{U}(0, 1), \quad i = 1, 2, 3, 4 \quad (8)$$

then values need to be normalized so that the sum of all four values is equal to one:

$$v_i = \frac{u_i}{\sum_{j=1}^4 u_j}, \quad i = 1, 2, 3, 4 \quad (9)$$

The complete list of values, `all_values`, is created by concatenating `v`’s with the scaled values:

$$\text{all_values} = [v_1, v_2, v_3, v_4] \quad (10)$$

3.7.2 Condition Check for Validity. The difference between opposite actions must satisfy a minimum threshold set by the *margin value* parameter. This parameter guarantees that the possibilities are distant enough to prevent actions from canceling each other. If this condition is not met, the generation process restarts. The margin value used is described in Sec. 4.1.4:

$$|v_1 - v_3| \geq \text{margin} \quad (11)$$

3.7.3 Generating the PDL_{4×4} Matrix. The next step is to create a 4×4 matrix, P' . Each row i is a cyclic permutation (shift) of *all_values* by i positions:

$$P' = \begin{bmatrix} v_1 & v_2 & v_3 & v_4 \\ v_4 & v_1 & v_2 & v_3 \\ v_3 & v_4 & v_1 & v_2 \\ v_2 & v_3 & v_4 & v_1 \end{bmatrix} \quad (12)$$

In general, each element in row i and column j of P' is defined as

$$P'[i, j] = v_{((j+i) \bmod 4) + 1} \quad (13)$$

3.7.4 Generating the PDL_{4×6} Matrix. To have the PDL matrix represent the probabilities for an action space of 6, a 4×6 matrix, P , is extended using specific transformations of P' . The columns of P are populated as follows:

$$P[i, j] = \begin{cases} P'[i, j] & j = 0 \\ P'[i, j-1] & j = 3 \\ 0.5 \times P'[i, \text{ceil}(j/2)] & j = 1, 2 \\ 0.5 \times P'[i, \text{ceil}(j+1)/2] & \text{otherwise} \end{cases} \quad (14)$$

3.7.5 Normalizing Rows in P . Each row in P is normalized to ensure the values sum to 1.

$$\text{row_sums}[i] = \sum_j P[i, j] \quad (15)$$

$$P[i, j] = \frac{P[i, j]}{\text{row_sums}[i]} \quad (16)$$

Four valid PDLs are generated: P . The total number of PDLs generated is outlined in Sec. 4.1.3.

4 Experiments and Results

4.1 Box Movement Distribution and Fitness Evaluation Criteria. At each step, agents will simultaneously push the box after determining what action to take. In one simulation, the displacement of the box in both the x - and y -axes in which the box is pushed from its starting point is recorded. Each simulation is exactly one step long. After a significant amount of simulations, we can plot out the resulting displacement distribution of the box in a kernel density estimate (KDE) plot. We can call the resulting KDE plot as box movement distribution (BMD) plot. Figure 5 shows an example of BMD.

When using the map, agents should be able to collaborate efficiently. To be considered effective, the map needs to pass a fitness test, which consists of an origin avoidance test and a uniformity of angular spread test. We must ensure that BMD plot demonstrates that agents minimize the amount of conflicting forces exerted while pushing the box in all directions with equal likelihoods.

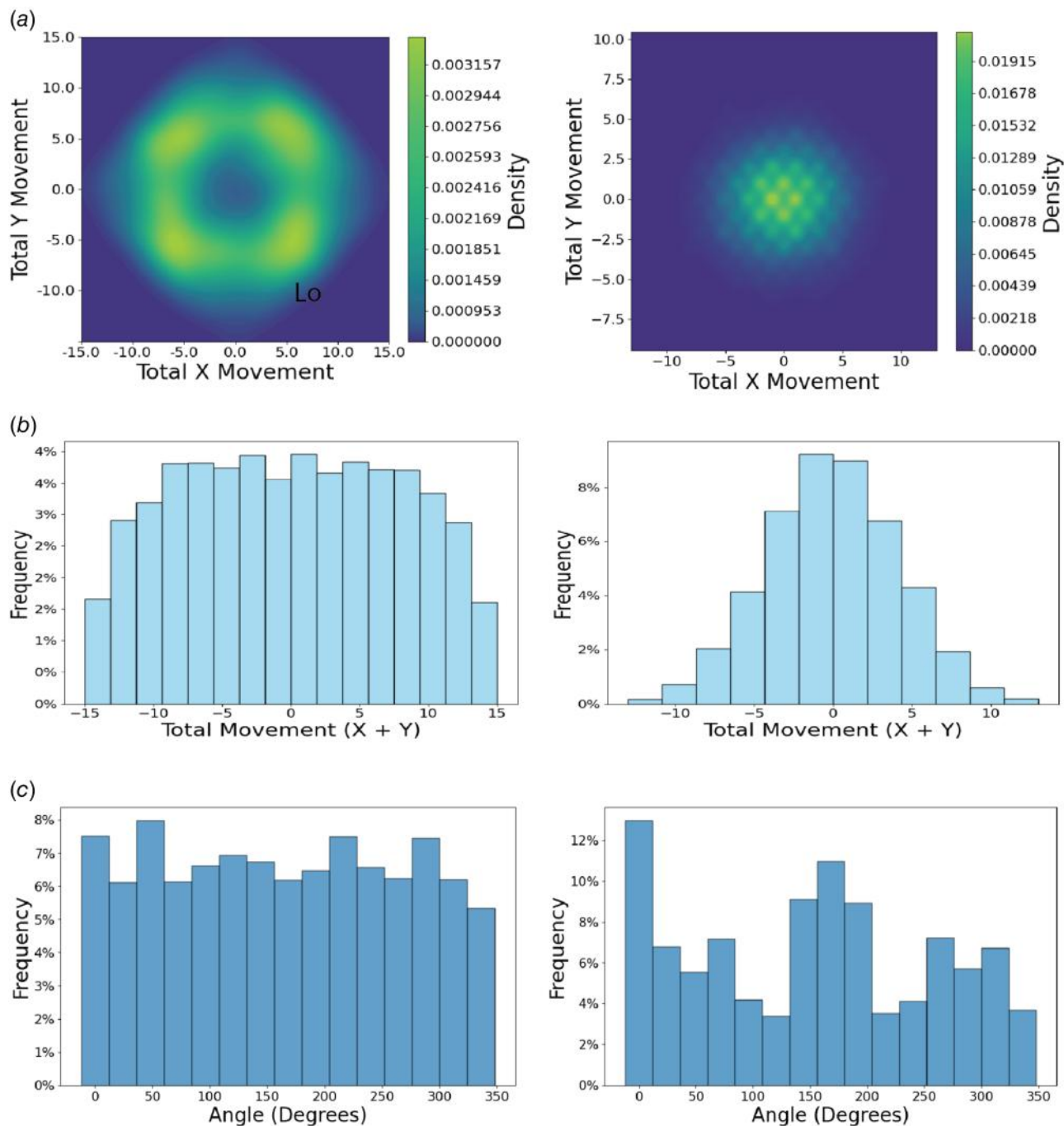


Fig. 5 Comparison of SPI and random exploration. Each subfigure compares a different metric of box movement.

4.1.1 Origin Avoidance. The first component of the fitness test evaluates origin avoidance, which measures the percentage of time the agents can displace the box a sufficient distance. The origin avoidance fitness score is calculated as follows:

$$\text{origin_avoidance_fitness} = 1 - \left(\frac{\text{near_origin_count}}{\text{total_count}} \right) \quad (17)$$

In the tests, the box is considered near the origin if its total displacement is three times less than its maximum potential displacement. This ensures the agents cooperate so that the box is sufficiently displaced from its original position. The count of instances where the box remains near the origin is recorded as *near_origin_count*, while *total_count* denotes the overall count of test instances. A low origin avoidance fitness score would

indicate that the agents constantly exert conflicting forces, unable to consistently displace the box far from the origin. On the other hand, a high origin avoidance fitness score would indicate that agents are generally able to collaborate, as they rarely exert conflicting forces.

4.1.2 Uniformity of Angular Spread. The second component of the fitness test evaluates the uniformity of the box's angular spread, which tests if the box can be pushed in any direction with equal likelihood. To calculate the uniformity of angular spread, the angle of each box from the origin is computed. Then, we define bin edges for n equally sized bins, each representing $360/n$ degrees, where n is the number of agents. Next, we compute the histogram of the angles (see Fig. 5) and utilize the mean and standard deviation of the histogram to determine the

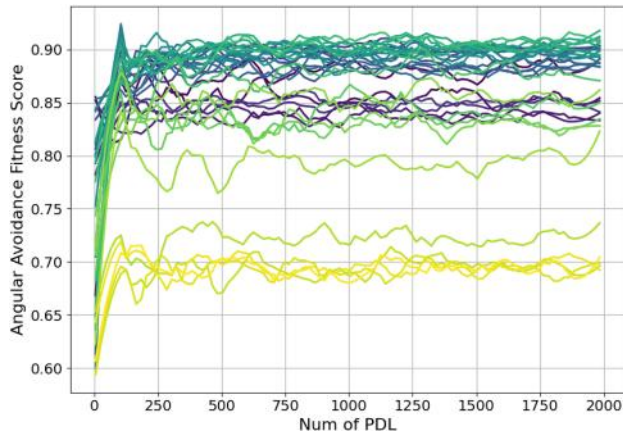


Fig. 6 The angular avoidance fitness score is tested for different margin and cap values. The lighter lines represent a high margin and cap value. The darker lines represent a low margin and cap value.

coefficient of variation (CV). A lower CV indicates higher uniformity in angular distribution. Therefore, we assess the uniformity of the angular spread by inverting the CV and normalizing it to [0, 1] range, as shown in the following equation:

$$\text{uniformity_of_angular_spread} = \frac{1 - \text{CV}}{\text{CV}} \quad (18)$$

A low fitness score indicates insufficient diversification in the box's movement directions, which can hinder the training process. Conversely, a higher score reflects that, throughout the simulations, the agents have successfully aligned themselves to enable multidirectional pushing of the box. Such flexibility in angular spread is essential for maximizing the range of the box's potential movement.

4.1.3 Number of Probability Distribution Lists Generated. When there are 15 agents, the angular avoidance fitness score increases as the number of PDL distributions increases to approximately 250 PDLs. Beyond this point, the fitness score plateaus, indicating that additional increases in the number of PDLs do not provide a significant additional benefit to angular avoidance. Figure 6 corroborates this finding, showing the same threshold behavior across different margin and cap configurations. Therefore, in the experiment, 4000 PDLs are used to guarantee maximal performance regarding angular avoidance fitness

4.1.4 Generating Margin Values. In the experiment, 15 agents and 4000 PDLs are used. The optimal margin values were systematically tested to evaluate their impact on performance metrics. A total of 100,000 simulations were conducted and the results were analyzed using a line graph.

As margin value increases, the origin avoidance fitness score also improves. An increase in the margin value up to 0.3 enhances the fitness score. Beyond this threshold, further increases in margin value result in a sharp decline in angular spread fitness. Based on these observations, a margin value of 0.3 was selected for the experiment.

However, multiple parameter configurations exist where both the origin avoidance and angular spread criteria are satisfactorily met. The choice of parameter values ultimately depends on the experimenter's priority [22] whether greater emphasis should be placed on origin avoidance or angular spread optimization.

4.1.5 Fitness Test—SPI-Guided Versus Random Exploration. The traditional random exploration method yields an origin avoidance score of 0.215 and an angular spread score of 0.714. In contrast, the proposed SPI-based method achieves a significantly higher origin avoidance score of 0.899 and an angular spread score of 0.911. By comparing the differences between both methods using the box movement graph and histogram of point directions, it is evident that SPI enables the box to achieve a wider range of movement while ensuring more uniform exploration across all directions (see Fig. 5).

4.2 Experiment Results and Discussion. To evaluate the effectiveness of SPI, we conducted comprehensive experiments examining multiple variables including team size, initialization parameters, and speed factors. All experimental conditions were averaged across 100 independent trials with randomized seeds to ensure statistical robustness and generalizability of results. Performance was assessed using four key metrics: episode success rate (ESR), number of steps (NoS), cumulative reward, and computational training time. These metrics were systematically analyzed and visualized to facilitate quantitative comparison between SPI-guided and baseline random exploration approaches. Figure 7 demonstrates some examples that our agents faced.

4.2.1 Episodic Success Rate. In this study, we define "success" as the scenario in which the box reaches the goal without colliding with any obstacles. The ESR quantifies the proportion of successful episodes throughout the training process. It can provide a measure of learning effectiveness over time.

Figure 8 demonstrates that ESR increases progressively throughout training for both SPI and random exploration methods. While both approaches exhibit comparable initial improvement rates, SPI-based training achieves a steeper learning curve and converges to superior final performance levels. These findings confirm that SPI effectively enhances team coordination and learning efficiency. However, as team size increases, the performance advantage of SPI over random exploration diminishes. This reduction in effectiveness can be attributed to the increased likelihood of agents selecting identical actions when using SPI-based probability distributions, resulting in unbalanced force application that increases the probability of collision with environmental obstacles.

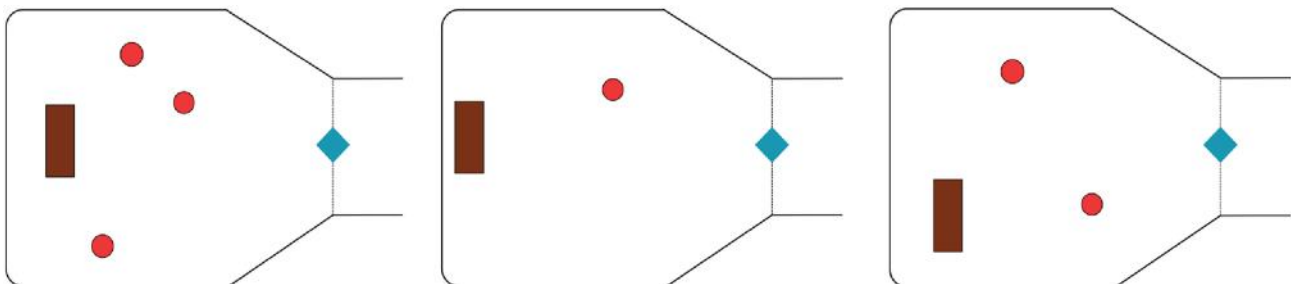


Fig. 7 Examples of training environments

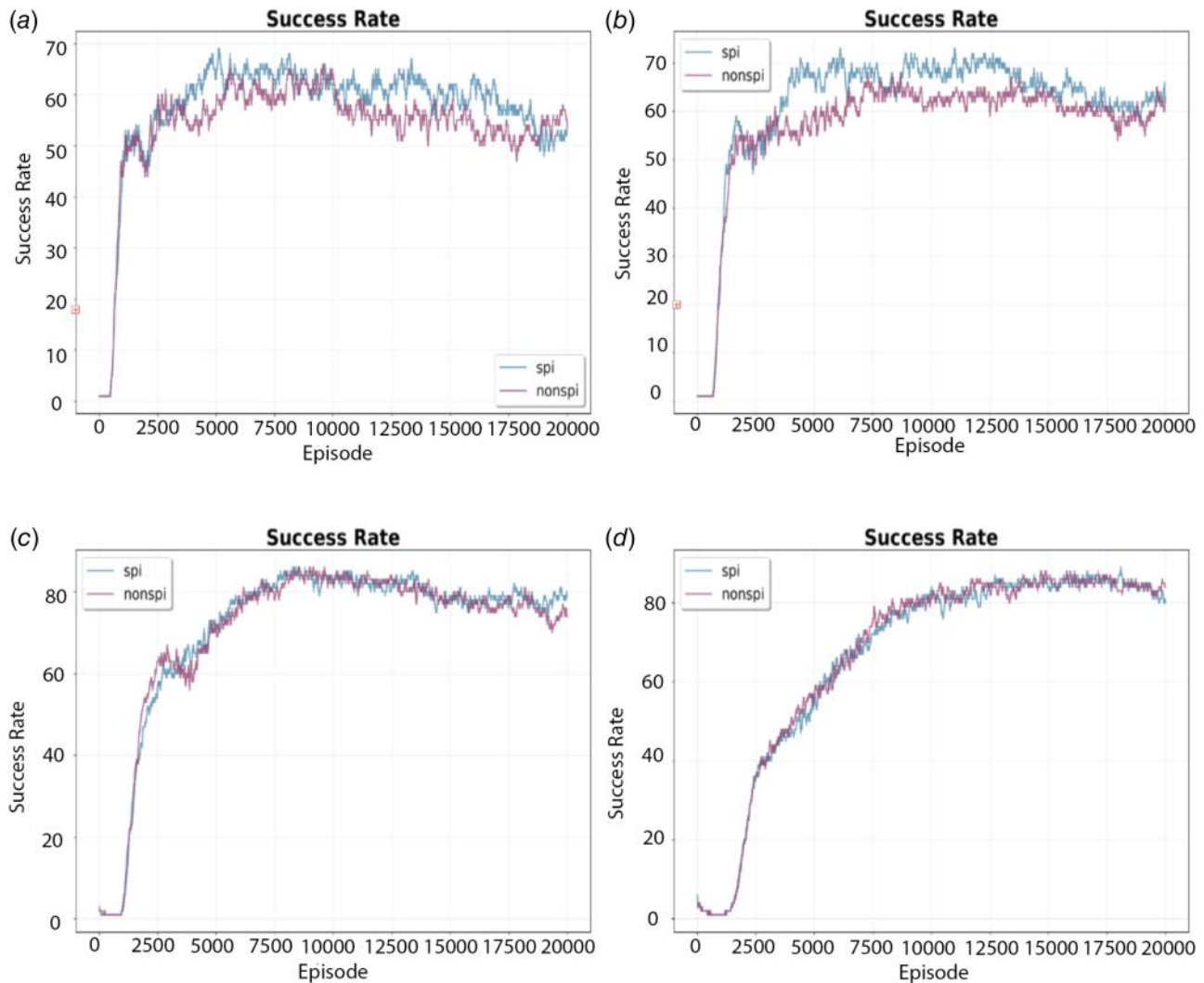


Fig. 8 Success rate comparisons. SPI (blue) versus Non-SPI (purple).

4.2.2 Number of Steps. The NoS metric quantifies the total time-steps required for a team to complete a single episode within the MDP framework. Although SPI and random exploration converge to similar final cumulative rewards, their convergence speeds differ. NoS is used as the primary metric because it directly reflects policy efficiency. As shown in the results, SPI achieves lower NoS and reduced training time, indicating faster convergence to more efficient policies even when final rewards are comparable.

We monitored NoS values throughout the training process, with results presented in Fig. 9. The figure shows the distribution of episode completion times, where the x -axis represents the number of steps required to successfully transport the box to its destination, and the y -axis indicates the frequency of episodes completed within each step range.

The results demonstrate that SPI-based teams require fewer steps to complete the task compared to the random exploration method. This reduction indicates that SPI-guided agents discover more efficient trajectories during training and converge to superior policies. These findings have substantial implications for engineering applications, as the decrease in NoS directly correlates with reduced energy consumption and operational costs. Additionally, our analysis reveals that optimal performance is achieved with a team composition of 15 agents, suggesting that this configuration represents the ideal balance between coordination efficiency and redundant agent deployment for the given experimental parameters.

4.2.3 Rewards. Team performance is evaluated by monitoring the cumulative rewards achieved by the teams using both methods, with results presented in Fig. 10. The results show that SPI and random exploration methods achieve comparable final reward values, with neither approach demonstrating a significant advantage in terms of ultimate performance. This outcome aligns with the design objective of SPI, which targets training efficiency rather than maximum rewards. Both methods eventually converge to successful task completion, with the primary distinction being the rate of convergence rather than final performance quality.

4.2.4 Computational Training Time. To evaluate the computational efficiency of SPI, we conducted a comprehensive performance assessment comparing models trained with and without the proposed method (see Table 3). All experiments were performed on compute nodes equipped with AMD EPYC 7513 processors (Zen 3 microarchitecture, 2.60 GHz base frequency), each containing 64 CPU cores and 248 GB of system memory. Training times were averaged over 100 random seeds to ensure robust statistical estimates and minimize variance in performance measurements.

Our experimental results reveal that SPI implementation yields a consistent reduction in training time, achieving improvements of 3.5–4.8% in randomized environments. This performance enhancement supports our hypothesis that SPI facilitates more directed exploration by enabling agents to pursue

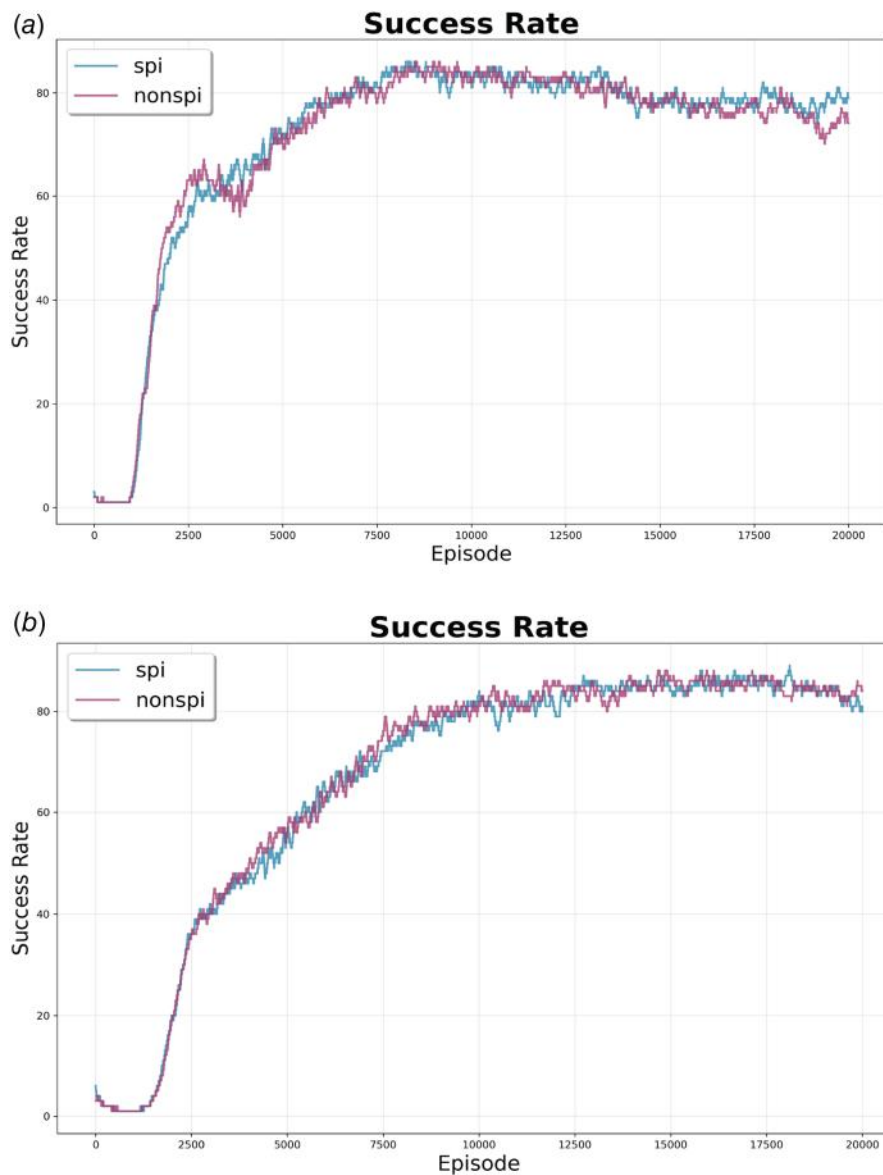


Fig. 9 NoS in teams of 15 agents (a) and 60 agents (b). SPI (blue) versus Non-SPI (purple).

Multi-Agent Box Pushing: Average Reward Comparison (agents_15)

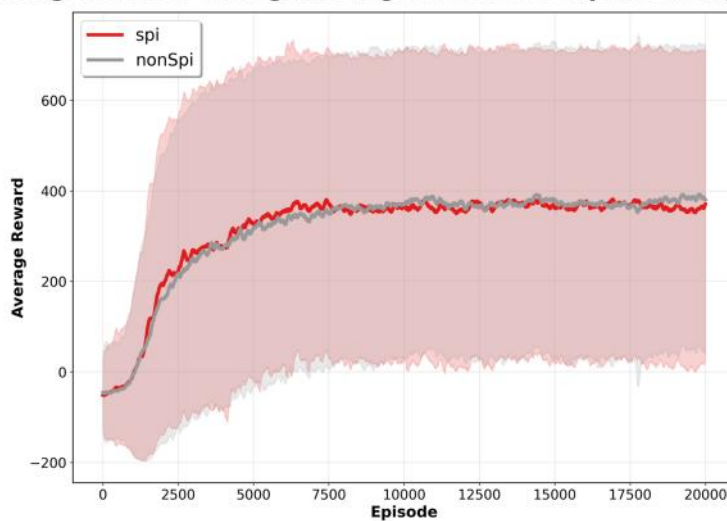


Fig. 10 Cumulative rewards

Table 3 Computational training time (s)

Num agents	Fixed env		Random env	
	SPI	Non-SPI	SPI	Non-SPI
8	21,894	23,493	24,830	25,993
15	22,211	23,942	28,212	29,267
30	22,979	25,226	30,329	31,437
60	27,961	29,682	31,430	32,997

strategically advantageous trajectories, thereby reducing the computational overhead associated with undirected exploration behaviors.

Overall, the results indicate that SPI primarily improves learning efficiency. While both SPI and non-SPI methods reach similar cumulative rewards, SPI leads to consistently better team coordination during training, evidenced by reduced NoS, increased success rates in the early and middle stages, and shorter overall computational training time. These results suggest that SPI shapes how agents learn to collaborate, leading to more efficient trajectories and coordinated behaviors.

5 Conclusions

This study introduces the SPI as an implicit coordination mechanism designed to enhance exploration efficiency in MARL without requiring direct inter-agent communication. The efficacy of SPI was implemented and assessed through box-pushing scenarios that demand robust collective coordination. Experimental results reveal that SPI substantially improves both agent coordination and training efficiency while eliminating communication overhead.

Teams employing SPI exhibited more effective group behaviors, including higher ESR, fewer NoS, and superior trajectory optimization. It is indicated that the SPI enables agents to identify more optimal trajectories by minimizing conflicting behaviors and facilitating balanced exploration across all spatial dimensions. Moreover, the results indicate that SPI accelerates learning convergence and reduces redundant actions in both successful and unsuccessful task attempts. These qualities position SPI as a viable solution for real-world multiagent robotic deployments, especially in applications where explicit communication channels are constrained or unavailable, such as underwater vehicle operations.

Despite these promising outcomes, the study acknowledges limitations stemming from the simulations and specific cases. Future investigations will examine adaptive SPI architectures incorporating dynamic probability distributions to further refine agent interaction patterns. Additionally, subsequent research will focus on integrating systematic environmental factors into more sophisticated algorithmic structures, enabling SPI adaptation to volatile and unpredictable operational conditions.

Conflict of Interest

There are no conflicts of interest.

Data Availability Statement

The datasets generated and supporting the findings of this article are obtainable from the corresponding author upon reasonable request.

References

- [1] Gorodetskii, V. I., 2012, "Self-Organization and Multiagent Systems: I, Models of Multiagent Self-Organization," *J. Comput. Syst. Sci. Int.*, **51**(2), pp. 256–281.
- [2] Huang, B., and Jin, Y., 2023, "Social Learning in Self-Organizing Systems for Complex Assembly Tasks," *Adv. Eng. Inform.*, **57**(1), p. 102109.
- [3] Li, Y., Zheng, Y., and Yang, Q., 2019, "Efficient and Effective Express via Contextual Cooperative Reinforcement Learning," *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Anchorage, AK, Aug. 4–8, pp. 510–519.
- [4] Liu, W., Zhuang, P., Liang, H., Peng, J., and Huang, Z., 2018, "Distributed Economic Dispatch in Microgrids Based on Cooperative Reinforcement Learning," *IEEE Trans. Neural Netw. Learn. Syst.*, **29**(6), pp. 2192–2203.
- [5] de Witt, C. A. S., Foerster, J. N., Farquhar, G., Torr, P. H. S., Böhrer, W., and Whiteson, S., 2019, "Multi-Agent Common Knowledge Reinforcement Learning," *Advances in Neural Information Processing Systems*, 32, Vancouver, BC, Canada, Dec. 8–14, pp. 9904–9915.
- [6] Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A. N., and Bonawitz, K., 2021, "Advances and Open Problems in Federated Learning," *Foundations and Trends in Machine Learning*, **14**(1–2), pp. 1–210.
- [7] Huang, B., Hao, J., and Jin, Y., 2024, "Impact of Social Learning on Teamwork Efficiency in Learning-Based Self-Organized Systems," *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Washington, DC, Aug. 25–28.
- [8] Claus, C., and Boutillier, C., 1998, "The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems," *Proceedings of the National Conference on Artificial Intelligence (AAAI)*, Madison, WI, July 26–30, AAAI Press, pp. 746–752.
- [9] Hurtado, L. A., Mocanu, E., Nguyen, P. H., Gibescu, M., and Kamphuis, R. I. G., 2018, "Enabling Cooperative Behavior for Building Demand Response Based on Extended Joint Action Learning," *IEEE Trans. Industr. Inform.*, **14**(1), pp. 127–136.
- [10] Oroojlooy, A., and Hajinezhad, D., 2023, "A Review of Cooperative Multi-Agent Deep Reinforcement Learning," *Appl. Intell.*, **53**(11), pp. 13677–13722.
- [11] Wang, Y., and De Silva, C. W., 2006, "Multi-Robot Box-Pushing: Single-Agent Q-Learning Versus Team Q-Learning," 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems, Beijing, China, Oct. 9–15, pp. 3694–3699.
- [12] Rahimi, M., Gibb, S., Shen, Y., and La, H. M., 2019, "A Comparison of Various Approaches to Reinforcement Learning Algorithms for Multi-Robot Box Pushing," *Advances in Engineering Research and Application*, H. Fujita, D. C. Nguyen, N. P. Vu, T. L. Banh, H. H. Puta, eds., Springer International Publishing, Cham, pp. 16–30.
- [13] Parra-Gonzalez, E. F., Ramirez-Torres, J. G., and Toscano-Pulido, G., 2009, "A New Object Path Planner for the Box Pushing Problem," 2009 Electronics, Robotics and Automotive Mechanics Conference (CERMA), Cuernavaca, Morelos, Mexico, Sept. 29–Oct. 2, pp. 119–124.
- [14] Watkins, C. J., and Dayan, P., 1992, "Q-Learning," *Mach. Learn.*, **8**(3–4), pp. 279–292.
- [15] Hwang, K. S., Ling, J. L., and Wang, W. -H., 2014, "Adaptive Reinforcement Learning in Box-Pushing Robots," 2014 IEEE International Conference on Automation Science and Engineering (CASE), Taipei, Taiwan, Aug. 18–22, pp. 1182–1187.
- [16] Yasuda, T., Ohkura, K., and Yamada, K., 2013, "Multi-Robot Cooperation Based on Continuous Reinforcement Learning With Two State Space Representations," 2013 IEEE International Conference on Systems, Man, and Cybernetics, Manchester, UK, Oct. 13–16, pp. 4470–4475.
- [17] Wang, Y., and de Silva, C., 2005, "An Object Transportation System With Multiple Robots and Machine Learning," *Proceedings of the 2005 American Control Conference*, Portland, OR, June 8–10, Vol. 2, pp. 1371–1376.
- [18] Amato, C., 2024, "An Introduction to Centralized Training for Decentralized Execution in Cooperative Multi-Agent Reinforcement Learning," 2409.03052, <https://arxiv.org/abs/2409.03052>
- [19] Fan, W., Yu, Z., Ma, C., Li, C., Yang, Y., and Zhang, X., 2024, "Towards Efficient Collaboration via Graph Modeling in Reinforcement Learning," 2410.15841, <https://arxiv.org/abs/2410.15841>
- [20] Wang, J., Ye, D., and Lu, Z., 2023, "More Centralized Training, Still Decentralized Execution: Multi-Agent Conditional Policy Factorization," 2209.12681, <https://arxiv.org/abs/2209.12681>
- [21] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., and Mordatch, I., 2020, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," 1706.02275, <https://arxiv.org/abs/1706.02275>
- [22] Foerster, J. N., Farquhar, G., Afouras, T., Nardelli, N., and Whiteson, S., 2018, "Counterfactual Multi-Agent Policy Gradients," *Proceedings of the AAAI Conference on Artificial Intelligence*, New Orleans, LA, Feb. 2–7, AAAI Press, pp. 2974–2982.
- [23] Kim, D., Moon, S., Hostallero, D., Kang, W. J., Lee, T., Son, K., and Yi, Y., 2019, "Learning to Schedule Communication in Multi-Agent Reinforcement Learning," 1902.01554, <https://arxiv.org/abs/1902.01554>
- [24] Chen, G., 2020, "A New Framework for Multi-Agent Reinforcement Learning – Centralized Training and Exploration With Decentralized Execution via Policy Distillation," *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, Richland, SC, Auckland, New Zealand, May 9–13, pp. 1801–1803.
- [25] Vezhnevets, A. S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., and Kavukcuoglu, K., 2017, "FeUdal Networks for Hierarchical Reinforcement Learning," *Proceedings of the 34th International Conference on Machine Learning*, Sydney, Australia, Aug. 6–11, pp. 3540–3549.
- [26] Shin, Y., Seo, S., Yoo, B., Kim, H., Song, H., and Lee, S., 2023, "Survey on Recent Advances in Multiagent Reinforcement Learning Focusing on Decentralized Training With Decentralized Execution Framework," *Electron. Telecommun. Trends*, **38**(4), pp. 95–103.
- [27] Bellman, R., 1957, "A Markovian Decision Process," *J. Math. Mech.*, **6**(5), pp. 679–684.

- [28] Matignon, L., Jeanpierre, L., and Mouaddib, A.-I., 2012, "Distributed Value Functions for Multi-Robot Exploration," 2012 IEEE International Conference on Robotics and Automation, Saint Paul, MN, May 14–18, pp. 1544–1550.
- [29] Wolpert, D., Wheeler, K., and Tumer, K., 1999, "General Principles of Learning-Based Multi-Agent Systems," arXiv: Multiagent Systems.
- [30] Peshkin, L., Kim, K.-E., Meuleau, N., and Kaelbling, L. P., 2000, "Learning to Cooperate via Policy Search," Proceedings of the 16th Conference on Uncertainty in Artificial Intelligence, Stanford, CA, June 30–July 3, pp. 489–496.
- [31] Guestrin, C., Koller, D., and Parr, R., 2001, "Multiagent Planning With Factored MDPs," Proceedings of the 14th International Conference on Neural Information Processing Systems: Natural and Synthetic, Whistler, British Columbia, Canada, Dec. 3–8, pp. 1523–1530.
- [32] Szer, D., and Charpillet, F., 2004, "Improving Coordination With Communication in Multi-Agent Reinforcement Learning," 16th IEEE International Conference on Tools with Artificial Intelligence, Boca Raton, FL, Nov. 15–17, pp. 436–440.
- [33] Li, S., Gupta, J. K., Morales, P., Allen, R., and Kochenderfer, M. J., 2021, "Deep Implicit Coordination Graphs for Multi-Agent Reinforcement Learning," arXiv.
- [34] Huang, B., and Jin, Y., 2022, "Reward Shaping in Multiagent Reinforcement Learning for Self-Organizing Systems in Assembly Tasks," *Adv. Eng. Inform.*, **54**(1), p. 101800.
- [35] Team, P., 2009, Pygame.
- [36] Ji, H., and Jin, Y., 2022, "Knowledge Acquisition of Self-Organizing Systems With Deep Multiagent Reinforcement Learning," *ASME J. Comput. Inf. Sci. Eng.*, **22**(2), p. 021010.